

Chuqur o'rganish usullari

2-mavzu. Backpropagation, softmax va optimizatorlar — Practice

Mavzu: hisoblash grafi va zanjir qoidasi, reverse-mode autograd, softmax va cross-entropy, backpropagation ning matritsa shakli, gradient tekshiruv, gradientning so'nishi va portlashi, gradient clipping, dropout, mini-batch shovqini, optimizatorlar (SGD, Momentum, Adam, AdamW) va qadam jadvallari (warmup, cosine).

Maqsad: (1) grafda va matritsa ko'rinishida gradientlarni qo'lda hisoblash; (2) asosiy formulalarni isbotlash; (3) backpropagation, autograd va optimizatorlarni NumPy'da yozib, PyTorch bilan solishtirish; (4) o'qitishdagi tipik xatolarni aniqlash.

Baholash: har bir topshiriq 1 ball, jami 30 ball. Hisob topshiriqlarida javob 4 ta o'nlik xona aniqlikda; kod topshiriqlarida (17–30) kod xatosiz ishlashi, chop etilgan natija ko'rsatilgan formatda bo'lishi va 1–2 jumlati xulosa talab qilinadi.

Muhit: Python 3.11/3.12, `pip install -r requirements.txt` (versiyalar qat'iy: numpy 2.4.4, scikit-learn 1.8.0, torch 2.14.0, faqat CPU). Ma'lumotlar `sklearn.datasets` ichida — internet shart emas. Kod bitta `solutions.py` faylida `task_17()`, ..., `task_30()` funksiyalari sifatida yoziladi. Hujjatlar: <https://pytorch.org/docs/stable/optim.html>, <https://pytorch.org/docs/stable/autograd.html>, <https://pytorch.org/docs/stable/nn.html>.

A. Qo'lda hisoblash, isbot va tushunchalar (1–16)

Belgilar ma'ruzadagidek: qatorlar — namunalar, $Z^{(i)} = A^{(i-1)}W^{(i)} + \mathbf{1}b^{(i)T}$, $\delta^{(i)} = \partial L / \partial Z^{(i)}$, $\bar{v} = \partial L / \partial v$.

- (hisob) $f(x, y, z) = (x + y) \cdot \max(y, z)$, $x = 1$, $y = 3$, $z = 2$. Hisoblash grafini chizing (oraliq tugunlar $a = x + y$, $m = \max(y, z)$), oldinga yurishni bajaring va teskari tartibda $\bar{a}$, $\bar{m}$, $\bar{x}$, $\bar{y}$, $\bar{z}$ ni toping. Nega $\bar{y}$ ikki hissadan iborat?
- (hisob) Logitlar $\mathbf{z} = (1; 2; 0,5)$, to'g'ri sinf — ikkinchi. $\mathbf{p} = \text{softmax}(\mathbf{z})$, cross-entropy $\ell = -\ln p_y$ va $\partial \ell / \partial \mathbf{z}$ ni toping. $\mathbf{z} + 100 \cdot \mathbf{1}$ uchun $\mathbf{p}$ qanday o'zgaradi?
- (isbot, hisob) $\partial p_i / \partial z_j = p_i(\delta_{ij} - p_j)$, ya'ni $J = \text{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}^T$ ekanini isbotlang; J simmetrik va $J\mathbf{1} = \mathbf{0}$ ekanini ko'rsating. 2-topshiriqdagi $\mathbf{p}$ uchun J ning diagonal elementlarini va J_{12} ni hisoblang.
- (isbot) Softmax va cross-entropy uchun $\partial \ell / \partial \mathbf{z} = \mathbf{p} - \mathbf{y}$ ni isbotlang. $K = 2$ da $p_1 = \sigma(z_1 - z_2)$ ekanini va natija binary cross-entropy gradientiga $(p - y)$ aylanishini ko'rsating. Tekshiring: $\mathbf{z} = (1,5; -0,5)$.
- (isbot, hisob) $Z = AW + \mathbf{1}b^T$ va $\delta = \partial L / \partial Z$ bo'lsin. $dL = \langle \delta, dZ \rangle = \text{tr}(\delta^T dZ)$ dan foydalanib, $\nabla W = A^T \delta$, $\nabla b = \delta^T \mathbf{1}$, $\nabla A = \delta W^T$ ni isbotlang va o'lchamlarni tekshiring. Hisoblang: $A = \begin{pmatrix} 1 & 2 \\ 0 & -1 \end{pmatrix}$, $W = \begin{pmatrix} 1 & 0 & -1 \\ 2 & 1 & 0 \end{pmatrix}$, $\delta = \begin{pmatrix} 0,1 & -0,2 & 0,1 \\ 0,3 & 0 & -0,3 \end{pmatrix}$.
- (hisob) $n = 1$, $\mathbf{x} = (1, -1)$, $W^{(1)} = \begin{pmatrix} 1 & 2 \\ -1 & 1 \end{pmatrix}$, $\mathbf{b}^{(1)} = (0, -2)$, ReLU, $W^{(2)} = \begin{pmatrix} 1 & -1 \\ 2 & 0 \end{pmatrix}$, $\mathbf{b}^{(2)} = \mathbf{0}$, softmax + cross-entropy, to'g'ri sinf — ikkinchi. Oldinga yurish ($Z^{(1)}, A^{(1)}, Z^{(2)}, \mathbf{p}, L$), orqaga yurish ($\delta^{(2)}, \nabla W^{(2)}, \nabla \mathbf{b}^{(2)}, \delta^{(1)}, \nabla W^{(1)}, \nabla \mathbf{b}^{(1)}$) ni bajaring. $\eta = 0,1$ bilan bitta SGD qadamidan keyingi L ni toping. Qaysi gradientlar nol va nima uchun?
- (hisob, isbot) $f(t) = e^t$, $t = 0$, $\varepsilon = 0,1$. Markaziy $\frac{f(t+\varepsilon)-f(t-\varepsilon)}{2\varepsilon}$ va oldinga $\frac{f(t+\varepsilon)-f(t)}{\varepsilon}$ ayirmalarning xatosini hisoblang va Teylor formulasi bo'yicha bashorat ($\varepsilon^2/6$ va $\varepsilon/2$) bilan solishtiring. Umumiy xato $\frac{\varepsilon^2}{6}|f'''| + \frac{|u|f|}{\varepsilon}$ ($u = 1,1 \cdot 10^{-16}$) ni minimallashtiruvchi ε ni toping.
- (hisob) MLP 784–256–128–10, paket $n = 64$. (a) Parametrlar soni. (b) Oldinga yurishdagi matritsa ko'paytmalari FLOP soni ($\approx 2n \sum d_{i-1}d_i$) va bitta o'qitish qadami (≈ 3 baravar). (c) Sonli differensiallash bilan bitta gradient uchun nechta oldinga yurish kerak? (d) `float32` da parametrlar hajmi va orqaga yurish uchun saqlanadigan $X, Z^{(1)}, A^{(1)}, Z^{(2)}, A^{(2)}, Z^{(3)}$ hajmi (baytlarda).
- (hisob) (a) $L = 10$ qatlamli sigmoid tarmoqda $\|W^{(i)}\|_2 \leq 1$ bo'lsa, $\|\delta^{(1)}\|/\|\delta^{(10)}\|$ uchun yuqori chegara qancha? (b) ReLU va $\|W\|_2 = 1,5$ da o'sish ko'paytuvchisi $1,5^9$ ni hisoblang. (c) $\mathbf{g} = (3; 4; 12)$, gradient clipping $c = 5$: $\mathbf{g} \cdot \min(1, c/\|\mathbf{g}\|)$ ni toping.
- (hisob, isbot) Inverted dropout, `nn.Dropout(p=0.2)`, ya'ni $q = 0,8$. $\mathbf{a} = (1; 2; 3; 4; 5)$, niqob $\mathbf{m} = (1; 0; 1; 1; 0)$. $\bar{\mathbf{a}} = \mathbf{m} \odot \mathbf{a}/q$ ni va $\bar{\bar{\mathbf{a}}} = \mathbf{1}$ bo'lganda $\bar{\mathbf{a}}$ ni toping. $\mathbb{E}[\bar{a}_j] = a_j$ va $\text{Var}[\bar{a}_j] = a_j^2(1-q)/q$ ni isbotlang. Inferensiyada nima qilinadi?
- (isbot, tushuncha) $\mathbf{g}_B = \frac{1}{B} \sum_{k \in B} \mathbf{g}_k$, bunda indekslar qaytarib, tekis tanlanadi. $\mathbb{E}\mathbf{g}_B = \nabla L$ va $\text{Cov}(\mathbf{g}_B) = \frac{1}{B} \Sigma$ ni isbotlang. $B = 16$ dan $B = 256$ ga o'tilsa, shovqin standart chetlanishi necha marta kamayadi? "Linear scaling rule" bo'yicha $B = 256$, $\eta = 0,1$ dan $B = 1024$ ga o'tilganda η qancha olinadi?
- (hisob) $f(\boldsymbol{\theta}) = \frac{1}{2}(\theta_1^2 + 16\theta_2^2)$, $\boldsymbol{\theta}_0 = (1; 1)$, $\eta = 0,05$. SGD va Momentum ($\mu = 0,9$, $\mathbf{v}_0 = \mathbf{0}$, $\mathbf{v}_t = \mu\mathbf{v}_{t-1} + \mathbf{g}_t$)

bilan ikki qadam bajaring, $f(\theta_2)$ ni solishtiring. Yaqinlashish sharti, optimal doimiy η va κ ni toping.

13. (*hisob*) Adam ($\eta = 0,01$, $\beta_1 = 0,9$, $\beta_2 = 0,999$, $\epsilon = 10^{-8}$), $\theta_0 = \mathbf{0}$, gradientlar $\mathbf{g}_1 = (0,1; -2)$, $\mathbf{g}_2 = (0,3; -1)$. $\mathbf{m}_t, \mathbf{v}_t, \hat{\mathbf{m}}_t, \hat{\mathbf{v}}_t, \theta_t$ ni $t = 1, 2$ uchun toping. Nega birinchi qadam ikki koordinatada bir xil kattalikda?
14. (*isbot*) $\mathbf{m}_0 = \mathbf{0}$, $\mathbb{E}\mathbf{g}_k = \mathbf{g}$ bo'lsa, $\mathbb{E}\mathbf{m}_t = (1 - \beta_1^t)\mathbf{g}$ ni isbotlang. $1 - \beta_1^t$ ni $t = 1, 10$ va $1 - \beta_2^t$ ni $t = 1, 10, 1000$ uchun hisoblang. Qaysi tuzatma uzoqroq muhim?
15. (*hisob, tushuncha*) Bitta parametr $\theta_0 = 1$, yo'qotish gradienti doim 0, $\eta = 0,1$, $\lambda = 0,01$. (a) Adam + L_2 (Adam(weight_decay=0.01), ya'ni $\mathbf{g} \leftarrow \mathbf{g} + \lambda\theta$) va (b) AdamW ($\theta \leftarrow \theta - \eta\lambda\theta$, so'ng Adam qadami) bilan uch qadamdan keyingi θ ni toping. Nima uchun (a) da so'nish λ ga deyarli bog'liq emas?
16. (*amaliy holat: o'qitishni tashxirlash*) (a) Warmup + cosine: $t < T_w$ da $\eta_t = \eta_{\max}(t + 1)/T_w$, aks holda $\eta_t = \frac{\eta_{\max}}{2}(1 + \cos \frac{\pi(t - T_w)}{T - T_w})$; $\eta_{\max} = 0,1$, $T_w = 10$, $T = 100$. $\eta_0, \eta_4, \eta_9, \eta_{10}, \eta_{55}, \eta_{99}$ ni toping. Step decay ($\eta_0 = 0,1$, har 30 epoxada $\gamma = 0,1$): 65-epoxada η ? (b) 10 sinfli klassifikatsiyada quyidagi holatlarning sababi va yechimini yozing: (i) η 0,5 ga oshirilgach 30-qadamda yo'qotish nan; (ii) yo'qotish boshidan 2,3026 da turibdi; (iii) train yo'qotishi kamayadi, validatsiya yo'qotishi 15-epoxadan keyin o'sadi; (iv) accuracy o'sadi, lekin train yo'qotishi $\approx 1,46$ dan pastga tushmaydi.

B. Dasturlash (17–30)

Umumiy qoidalar. import numpy as np, torch, torch.nn as nn, torch.nn.functional as F; har bir model yaratilishidan oldin torch.manual_seed(42); NumPy uchun np.random.default_rng(42); boshida torch.set_num_threads(1) va torch.use_deterministic_algorithms(True). **Digits bo'linishi** (22–26): X, y = load_digits(return_X_y=True); train_test_split(X / 16.0, y, test_size=0.3, stratify=y, random_state=42); PyTorch uchun float32 va int64; paketlar: DataLoader(TensorDataset(Xtr, ytr), batch_size=32, shuffle=True, generator=torch.Generator().manual_seed(42)). Kod python -W error::DeprecationWarning solutions.py bilan xatosiz ishlashi kerak.

17. (*micro-autograd*) Skalyar class Value yozing: data, grad, __add__, __mul__, tanh() va topologik tartiblash bilan backward() (gradientlar += bilan to'planadi). $f = \tanh(xy + z) \cdot x$, $x = 0,5$, $y = -2$, $z = 1,5$ uchun f va $\partial f / \partial x, \partial f / \partial y, \partial f / \partial z$ ni (.6f) chop eting va float64 PyTorch autograd bilan solishtiring.
18. (*barqaror cross-entropy*) log_softmax(Z) (log-sum-exp hiylasi: $z - \max z$) va cross_entropy(Z, y) (o'rtacha) funksiyalarini yozing. $Z = [[1000, 1001, 1002], [-5, 0, 5]]$, $y = [2, 0]$ uchun sodda formula (np.errstate(all='ignore') ichida), barqaror formula va F.cross_entropy natijalarini hamda birinchi qator softmax ini (np.round(..., 4)) chop eting.
19. (*softmax Yakobiani*) $\mathbf{z} = \text{torch.tensor}([1.0, 2.0, 0.5], \text{dtype=torch.float64})$. torch.autograd.functional.jacobian bilan olingan Yakobianni $\text{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}^T$ bilan solishtiring. $\mathbf{p}, J$ (4 xona) va uchta tekshiruvni chop eting: moslik, simmetriya, $J\mathbf{1} = \mathbf{0}$.
20. (*NumPy MLP va gradient tekshiruvi*) class MLP(sizes, act, rng): og'irliklar rng.normal(0, np.sqrt(gain / d_in), (d_in, d_out)) (ReLU uchun gain=2, tanh uchun 1), siljishlar nol; forward(X) barcha $Z^{(i)}, A^{(i)}$ ni saqlaydi; loss(X, y) — 18-topshiriqdagi cross_entropy; backward(y) -> (gW, gb) — ma'ruzadagi Algoritm 1. Tekshiruv: rng = np.random.default_rng(42); X = rng.normal(size=(5, 4)); y = rng.integers(0, 3, 5); net = MLP([4, 6, 3], act='tanh', rng=rng). Har bir parametr elementi uchun markaziy ayirma ($\epsilon = 10^{-6}$), nisbiy xato $\max|g_a - g_s| / \max(\max|g_s|, 10^{-12})$ ni (.1e) va ok: rel < 1e-7 ni chop eting (tartib: W1, W2, b1, b2).
21. (*sonli hosila xatosi*) $f = \sin$, $t = 1$, $\epsilon = 10^{-k}$, $k = 1, \dots, 12$: markaziy va oldinga ayirmaning $|\cdot - \cos|$ xatosini (.2e) jadval qilib chop eting va eng yaxshi markaziy ϵ ni toping. Natijani 7-topshiriqdagi nazariya bilan izohlang.
22. (*amaliy holat: qo'lda yozilgan raqamlar, NumPy*) 20-topshiriqdagi MLP([64, 64, 10], act='relu', rng=rng), rng = np.random.default_rng(42) (xuddi shu rng keyin aralashtirish uchun ham). Mini-batch SGD: $B = 32$, $\eta = 0,1$, 20 epoxa; har epoxa boshida idx = rng.permutation(n), paketlar idx[s:s+32]. Har 5 epoxada train CE (butun train, .4f) va test accuracy ni chop eting.
23. (*optimizatorlar*) nn.Sequential(nn.Linear(64, 64), nn.ReLU(), nn.Linear(64, 10)), nn.CrossEntropyLoss(), 20 epoxa. To'rt optimizator: SGD(lr=0.1), SGD(lr=0.05, momentum=0.9), Adam(lr=1e-3), AdamW(lr=1e-3, weight_decay=1e-2); har biri uchun yangi model (torch.manual_seed(42)) va yangi DataLoader. Oxirgi train CE va test accuracy (.4f) ni chop eting va xulosa qiling.
24. (*xato: ikki marta softmax*) 23-topshiriqdagi model va Adam(lr=1e-3) bilan ikki variant: to'g'ri (logitlar) va oxiriga nn.Softmax(dim=1) qo'shilgan. Train CE va test accuracy ni chop eting. Ikkinchi variantda CE ning quyi chegarasi $\ln(e + 9) - 1$ ni isbotlang va hisoblang.
25. (*gradient accumulation*) torch.manual_seed(42); model = nn.Linear(64, 10); digits train'ning birinchi 32 namunasi. (a) Butun paket gradienti bilan 4 ta 8 lik mikro-paketda (loss / 4).backward() yig'ilgan gradientni solishtiring (< 1e-6). (b) zero_grad() siz ikki marta backward(): $\|\nabla\| / \|\nabla_{\text{to'g'ri}}\|$ (.4f).

26. (*gradient portlashi va clipping*) 10 ta `[nn.Linear(64, 64), nn.ReLU()] + nn.Linear(64, 10)`; yashirin qatlam og'irliklari `nn.init.normal_(w, 0.0, std)`, bias nol (yaratilishi bilan). To'liq paketli SGD `lr=0.1`, 100 qadam, har qadamda `total = nn.utils.clip_grad_norm_(params, max_norm)` (clippingsiz holatda `max_norm=float('inf')`). Uch variant: `std=0.25` clippingsiz; `std=0.25, max_norm=1.0`; He: `std=sqrt(2/64)` clippingsiz. Birinchi qadamdagi gradient normasi (9.3f), 100 qadamdan keyingi CE va train accuracy ni chop eting. Hujjat: https://pytorch.org/docs/stable/generated/torch.nn.utils.clip_grad_norm_.html.
27. (*dropout*) `inverted_dropout(a, p, rng) -> (out, mask)` (`mask = rng.random(a.shape) < 1 - p`) va `inverted_dropout_backward(grad_out, mask, p)` yozing. `rng = np.random.default_rng(42)`; `a = rng.normal(2.0, 1.0, size=(1000, 100))`, `p = 0.2`: o'chirilgan ulush, `mean(out)/mean(a)`, backward qiymatlari. So'ng `torch.manual_seed(42)`, `nn.Dropout(p=0.2)` ni `torch.ones(1000, 100)` ga qo'llab: nol ulushi, noldan farqli qiymatlar, gradient qiymatlari; `eval()` rejimida aynan birlik akslantirish ekanini tekshiring.
28. (*mini-batch shovqini*) Breast cancer: `train_test_split(X, y, test_size=0.3, stratify=y, random_state=42)`, train `StandardScaler().fit_transform`, $\tilde{X} = [X, 1]$. Logistik neyron $w = 0$ nuqtada bitta namuna gradienti $g_k = (0.5 - y_k)\tilde{x}_k$. `rng = np.random.default_rng(42)`; $B \in \{8, 32, 128\}$ uchun 2000 ta paket (`rng.integers(0, n, B)`, qaytarib). Koordinatalar bo'yicha o'rtacha std, `std * sqrt(B)` va `max|g_B - ∇L|` ni (.4f) chop eting.
29. (*optimizatorlarni noldan yozish*) Rosenbrock $f = (1 - \theta_1)^2 + 100(\theta_2 - \theta_1^2)^2$, $\theta_0 = (-1, 2; 1)$, 500 qadam. NumPy'da SGD ($\eta = 3 \cdot 10^{-3}$), Momentum ($\mu = 0.9$, $\eta = 3 \cdot 10^{-3}$) va Adam ($\eta = 0.1$, standart $\beta_1, \beta_2, \epsilon$) ni yozing; natijani `float64` dagi `torch.optim.SGD`, `SGD(momentum=0.9)`, Adam bilan solishtiring ($< 1e-10$). θ_{500} (.4f), f (.2e) va moslikni chop eting.
30. (*amaliy holat: tibbiy tashxis, early stopping*) Breast cancer: `train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)` (test), qolgan qismdan `test_size=0.25, stratify, random_state=42` (validatsiya); `StandardScaler` train'da. `torch.manual_seed(42)`; `nn.Sequential(nn.Linear(30, 32), nn.ReLU(), nn.Dropout(0.2), nn.Linear(32, 2))`; `AdamW(lr=1e-2, weight_decay=1e-2)`; `LambdaLR` bilan 16-topshiriqdagi warmup + cosine ($T_w = 5$, $T = 100$, har epoxada `sched.step()`); $B = 32$. Early stopping: validatsiya CE 10^{-4} dan ko'proq yaxshilanmasa sabr 10 epoxa, eng yaxshi `state_dict` tiklanadi. Chop eting: to'plam o'lchamlari, 0, 4, 5, 10-epoxalardagi lr (.6f), to'xtash epoxasi, eng yaxshi epoxa va val CE, test accuracy va F1-score (`pos_label=0, malignant`). Hujjat: <https://pytorch.org/docs/stable/optim.html#how-to-adjust-learning-rate>.